

Final Report

**Quantisation Effects in Parameter-Efficient Domain Adaptation:
A Matched LoRA/QLoRA Study on UK Employment-Law Text**

Gabriel Saban

**Submitted in accordance with the requirements for the degree of
BSc Computer Science with Artificial Intelligence**

2025/26

COMP3931 Individual Project

The candidate confirms that the following have been submitted.

Items	Format	Recipient(s) and Date
Final Report	PDF file	Uploaded to Minerva (06/05/26)
Link to online code repository	URL	Submitted on 06/05/26

The candidate confirms that the work submitted is their own and the appropriate credit has been given where reference has been made to the work of others.

I understand that failure to attribute material which is obtained from another source may be considered as plagiarism.

(Signature of Student) Gabriel Saban

Summary

Large language models often require adaptation before they perform well on specialist legal text, yet continued pretraining is expensive in both memory and computation. This dissertation studies that trade-off through domain-adaptive continued pretraining on a curated corpus of UK employment-law text, with the central comparison framed as LoRA on an unquantised **bf16** base model versus QLoRA on a 4-bit **NF4** base model under matched data and optimisation settings.

The project defines a controlled experimental protocol in which corpus, split, sequence length, optimisation budget, evaluation cadence, runtime policy, and training seeds are held fixed. It constructs a reportable UK employment-law corpus combining employment tribunal judgments with authoritative guidance and doctrine, then implements a reproducible LoRA/QLoRA training and reporting pipeline that records held-out perplexity, memory, runtime, and objective stability events.

The final intrinsic study comprises twelve completed **protocol_v3** runs: four reportable conditions repeated across three shared training seeds. Under the matched-rank comparison, LoRA **r16** achieves the best mean overall validation perplexity, while QLoRA **r16** substantially reduces training memory but incurs a large runtime penalty and does not match LoRA’s held-out quality. Within the quantised regime, increasing adapter rank from 16 to 32 recovers some intrinsic quality at little additional systems cost, whereas increasing further to 64 gives no meaningful overall gain. No final reportable condition records objective stability failures.

The dissertation therefore characterises QLoRA as a practical memory-saving intervention rather than a cost-free substitute for unquantised LoRA in this setting. Its main contribution is a controlled account of how 4-bit quantisation changes the quality–memory–runtime trade-off in parameter-efficient domain-adaptive continued pretraining on UK employment-law text.

Acknowledgements

I would like to thank my project supervisor, Dr Jonathan Cornford, for his guidance, feedback, and support throughout this dissertation. His advice was consistently helpful in shaping the experimental design, refining the research questions, and keeping the project focused on a rigorous and defensible comparison. I am especially grateful for the time he gave to discussing the work as it developed and for the clarity of his feedback at points where the project needed sharper direction.

I would also like to acknowledge the School of Computing and the University of Leeds for providing access to the cluster computing resources used to run the experimental workloads reported in this dissertation.

Contents

1	Introduction and Background Research	1
1.1	Research Problem, Aim, and Scope	1
1.1.1	Research Problem and Motivation	1
1.1.2	Aim, Objectives, and Research Questions	2
1.1.3	Scope and Boundaries	3
1.2	Conceptual Foundations	3
1.2.1	Transformer Language Models and Causal Language Modelling	3
1.2.2	Perplexity as an Intrinsic Evaluation Measure	4
1.3	Domain Adaptation and Efficient Adaptation Methods	4
1.3.1	Domain-Adaptive Continued Pretraining	4
1.3.2	Parameter-Efficient Fine-Tuning	5
1.3.3	LoRA	5
1.3.4	Quantisation and QLoRA	5
1.4	Evaluation Lenses and Legal-Domain Context	6
1.4.1	Quality, Efficiency, and Stability as Joint Evaluation Axes	6
1.4.2	Legal NLP and UK Employment-Law Text	6
1.4.3	Efficiency and Optimisation Trade-offs in Prior Work	7
1.5	Research Gap and Project Positioning	7
1.5.1	Research Gap and Project Positioning	7
2	Methodology	9
2.1	Study Design and Protocol Governance	9
2.1.1	Comparative Study Design	9
2.1.2	Protocol Evolution and Final Freeze	9
2.2	Experimental Conditions and Model Configuration	10
2.2.1	Run Matrix and Fairness Criteria	10
2.2.2	Base Model and Tokenizer	11
2.2.3	Adapter Configurations	11
2.3	Corpus Construction and Data Preparation	12
2.3.1	Corpus Design and Source Selection	12
2.3.2	Data Collection and Preprocessing	13

2.3.3	Tokenisation, Document Capping, and Corpus Freeze	13
2.3.4	Train–Validation Split and Leakage Control	13
2.3.5	Training Representation	14
2.4	Training Environment and Procedure	15
2.4.1	Hardware and Software Environment	15
2.4.2	Training Budget and Runtime Policy	15
2.4.3	Baseline Evaluation and Checkpoint Selection	16
2.5	Evaluation, Aggregation, and Validity	16
2.5.1	Outcome Measures	16
2.5.2	Aggregation and Analysis Plan	17
2.5.3	Reproducibility, Validity, and Limitations	17
3	Implementation and Validation	18
3.1	Implementation Architecture	18
3.1.1	Pipeline Decomposition and Deliverables	18
3.1.2	Configuration-Driven Execution	18
3.2	Data, Training, and Reporting Realisation	19
3.2.1	Corpus Preparation and Training-Data Packaging	19
3.2.2	Training, Evaluation, and Reporting Pipeline	19
3.2.3	Orchestration and Failure Handling	20
3.3	Validation and Verification	20
3.3.1	Validation Strategy	20
3.3.2	Smoke Tests, Feasibility Checks, and Pilot Evidence	21
3.3.3	Instrumentation Validation and Artefact Integrity	22
3.3.4	Engineering Challenges and Quality Controls	22
4	Results, Evaluation and Discussion	23
4.1	Reporting Basis and Evaluation Frame	23
4.1.1	Completed Run Set and Basis of Analysis	23
4.1.2	Baseline and Checkpoint-Selection Frame	23
4.2	Intrinsic Quality Results	24
4.2.1	Primary LoRA versus QLoRA Comparison	24
4.2.2	QLoRA Rank Sweep	25
4.2.3	Learning-Curve Behaviour	26
4.3	Efficiency and Stability Trade-Offs	27
4.3.1	Memory and Runtime Trade-Offs	27

4.3.2	Stability Outcomes	28
4.4	Discussion and Answers to the Research Questions	28
4.4.1	Answer to the Research Questions	28
4.4.2	Relation to Prior Literature	29
4.4.3	Limitations and Threats to Validity	29
4.5	Conclusions and Future Work	30
4.5.1	Chapter and Dissertation Conclusions	30
4.5.2	Future Work	30
References		31
Appendices		34
A Self-appraisal		34
A.1	Critical self-evaluation	34
A.2	Personal reflection and lessons learned	35
A.3	Legal, social, ethical and professional issues	36
A.3.1	Legal issues	36
A.3.2	Social issues	36
A.3.3	Ethical issues	37
A.3.4	Professional issues	37
B External Material		38

Chapter 1

Introduction and Background Research

Large language models are pretrained on broad corpora and therefore capture general linguistic regularities more readily than the specialised terminology, document structure, and reasoning style of any one professional domain. When such models are applied to legal text, and especially to a relatively narrow subdomain such as UK employment law, this mismatch becomes more than a stylistic issue: it affects how well the model represents recurrent legal concepts, procedural language, and the distribution of domain-specific phrases. At the same time, adapting large models remains expensive in both computation and memory, which makes resource-aware adaptation strategies practically important as well as technically interesting.

This dissertation examines that problem through a controlled comparison of Low-Rank Adaptation (LoRA) and Quantized LoRA (QLoRA) for domain-adaptive continued pretraining on a curated corpus of UK employment-law text. In practical terms, the project asks whether the memory saving from 4-bit QLoRA changes the modelling quality, runtime, or stability of adaptation when compared with LoRA under the same corpus and optimisation exposure. The chapter introduces the research problem, defines the dissertation’s aim and scope, and reviews the literature needed to justify the experimental design. The review is problem-oriented rather than chronological: it begins with the foundations of transformer language modelling, then considers domain adaptation, parameter-efficient adaptation, quantised PEFT, and finally the legal-domain context in which the study is situated.

1.1 Research Problem, Aim, and Scope

1.1.1 Research Problem and Motivation

Transformer language models have established a general framework for large-scale sequence modelling by replacing recurrence with attention-based architectures that scale effectively with data and compute [30]. More recent open model families, including Llama 3, have extended that paradigm to large autoregressive models suitable for continued pretraining and specialisation [12]. However, the success of broad pretraining does not remove the problem of domain mismatch. Gururangan et al. show that continued pretraining on in-domain text can materially improve downstream performance, even when the starting model has already been pretrained on very large corpora [13]. In the legal domain, Chalkidis et al. likewise show that additional domain-specific pretraining can produce meaningful gains over general-purpose models [5].

The difficulty is that continued pretraining at modern model scales is expensive. Full adaptation of a multi-billion-parameter causal language model requires substantial memory, long runtimes, and careful systems engineering. Parameter-efficient fine-tuning (PEFT) methods reduce that cost by keeping most of the base model frozen and training only a small subset of additional parameters [16, 19]. Quantised PEFT methods, most prominently QLoRA, reduce the

frozen-model memory footprint further by storing the base model in low precision while still training adapter parameters [9]. This makes larger models accessible on more modest hardware, but it also changes the numerical and systems behaviour of training.

That change motivates the dissertation’s central question. The issue is not merely whether quantised adaptation is possible. QLoRA has already shown that it is possible to fine-tune a 4-bit quantised model effectively in large-model settings [9]. The more relevant unanswered question for this project is how quantisation changes the trade-off between held-out modelling quality, memory use, runtime efficiency, and training stability. To make that comparison interpretable, the corpus, split, sequence length, optimiser settings, step budget, evaluation cadence, runtime policy, and training seeds are controlled; perplexity, memory, runtime, and stability are measured as outcomes. Without those controls, a better result could simply reflect more training, different batching, or a different data regime rather than quantisation itself.

The curated UK employment-law corpus provides a meaningful test bed for this question. For the purposes of the present study, it was designed to capture a narrower practical legal subdomain than all-purpose legal corpora, while still remaining varied enough to include adjudicative text, practical guidance, and doctrinal exposition. That makes it appropriate for domain-adaptive continued pretraining: the objective is to model the language of the domain more effectively, not to solve a single downstream benchmark task in isolation.

1.1.2 Aim, Objectives, and Research Questions

The dissertation aim is:

To quantify how 4-bit quantisation changes adaptation quality, memory use, runtime efficiency, and training stability in parameter-efficient domain-adaptive continued pretraining, by comparing LoRA and QLoRA under fixed data and optimisation budgets on UK employment-law text.

This aim is operationalised through four objectives:

1. to construct and freeze a UK employment-law corpus for domain-adaptive continued pretraining;
2. to implement a reproducible LoRA/QLoRA pipeline recording intrinsic quality, memory, runtime, and stability outcomes;
3. to compare LoRA and QLoRA under matched optimisation budgets on the fixed corpus;
4. to examine whether QLoRA adapter rank materially alters the observed trade-off pattern.

These objectives lead to the following research questions:

1. Under matched data and optimisation budgets, how does QLoRA compare with LoRA on held-out intrinsic adaptation quality?
2. What memory and runtime trade-offs accompany 4-bit quantisation in this setting?
3. Does 4-bit quantisation materially alter training stability when the corpus, split, schedule, and runtime policy are held fixed?
4. Does increasing QLoRA adapter rank recover held-out adaptation quality?

1.1.3 Scope and Boundaries

The dissertation is deliberately narrow in scope. It focuses on domain-adaptive continued pretraining of a single 8B-parameter base model on a single frozen UK employment-law corpus. The main comparison is LoRA versus QLoRA at matched adapter rank, with a secondary rank sweep inside the quantised setting. Evaluation is intrinsic and multi-objective, covering held-out perplexity, memory use, runtime behaviour, and objective stability events.

Several related topics are intentionally out of scope. The dissertation does not compare against full-model fine-tuning, does not attempt to benchmark general legal AI capability, does not conduct an extensive hyperparameter search, and does not treat downstream task accuracy as the sole or primary basis for judging adaptation. These boundaries are methodologically useful because they keep the study aligned with its title: the question is about quantisation effects in parameter-efficient domain adaptation under fixed data and training settings, not about the entire design space of legal-domain model improvement.

1.2 Conceptual Foundations

1.2.1 Transformer Language Models and Causal Language Modelling

Modern large language models are built on the transformer architecture introduced by Vaswani et al. [30]. The key conceptual shift is that token representations are updated through attention over other tokens in the sequence, rather than through recurrence. This makes the model highly parallelisable and allows it to scale effectively to very large training sets and parameter counts. Although the transformer family now includes many variants, the present dissertation is concerned with autoregressive causal language models, where the model predicts the next token given only the tokens that precede it.

This matters because domain-adaptive continued pretraining preserves the same basic objective used in original pretraining. Rather than creating a new supervised task, it continues optimisation of a causal language model on text drawn from the target domain. The research question is therefore not whether the model can learn a new label space, but whether further exposure to in-domain text changes its language-modelling behaviour in a way that is beneficial and resource-efficient.

1.2.2 Perplexity as an Intrinsic Evaluation Measure

Perplexity is a standard intrinsic measure for causal language models because it is the exponential form of the average next-token negative log-likelihood. Let $x_{1:T}$ be a held-out token sequence and let a causal language model with parameters θ assign each token probability $p_\theta(x_t \mid x_{<t})$. The average negative log-likelihood is:

$$\text{NLL}(x_{1:T}) = -\frac{1}{T} \sum_{t=1}^T \log p_\theta(x_t \mid x_{<t}).$$

Perplexity is then:

$$\text{PPL}(x_{1:T}) = \exp(\text{NLL}(x_{1:T})).$$

Lower perplexity therefore means that the model assigns higher average probability to the held-out tokens. For domain-adaptive continued pretraining, this is a natural intrinsic measure because the training objective and the evaluation objective are aligned: both assess next-token prediction on text from the target distribution.

Perplexity is nevertheless insufficient on its own. A lower perplexity does not guarantee better downstream legal reasoning, better factual reliability, or better behaviour on user-facing legal tasks. It is best interpreted here as an intrinsic measure of held-out modelling quality under the chosen objective. For that reason, the dissertation combines perplexity with memory, runtime, and stability measures rather than treating it as a complete measure of legal capability.

1.3 Domain Adaptation and Efficient Adaptation Methods

1.3.1 Domain-Adaptive Continued Pretraining

Adaptive pretraining literature provides the main motivation for this study. Gururangan et al. show that a second phase of pretraining on in-domain text can improve downstream performance even after broad initial pretraining [13]. The basic rationale is that broad pretraining may still underrepresent specialised vocabulary, syntax, and discourse patterns, so continued pretraining can shift the model toward the target distribution before supervised fine-tuning takes place.

In legal NLP, Chalkidis et al. reach a similar conclusion from a different direction: legal-domain adaptation is a meaningful modelling choice rather than a cosmetic modification of a general model [5]. More recently, Xie et al. show that efficient continual pretraining remains relevant for building stronger domain-specific LLMs in generative settings [31]. What this literature does not provide is a controlled intrinsic PEFT comparison in causal LLMs under matched data and optimisation settings.

1.3.2 Parameter-Efficient Fine-Tuning

Parameter-efficient fine-tuning methods respond to a practical problem: retraining all model parameters is often unnecessary and increasingly infeasible as models scale. Houlsby et al. show that inserting small trainable adapter modules into a frozen pretrained network can achieve strong transfer performance while updating only a small fraction of the original model parameters [16]. This establishes the broader PEFT principle on which later methods build: the base model remains largely fixed, while adaptation is concentrated in a small, trainable parameter subset.

PEFT is attractive on limited hardware because it reduces the optimizer state, parameter storage, and checkpoint footprint associated with adaptation. However, PEFT does not remove the need for careful comparison. Adapter-module methods, prompt- or prefix-style tuning, and adaptive-capacity variants all change different parts of the adaptation problem. This dissertation therefore narrows the design space to LoRA and QLoRA because they isolate the specific question of low-rank adaptation with and without 4-bit base-model quantisation. Even within that narrower family, published results may conflate method effects with differing adapter targets, ranks, schedules, or runtime choices, so meaningful comparison requires those design variables, together with data and optimisation budgets, to be controlled as tightly as possible [27].

1.3.3 LoRA

LoRA replaces a full weight update with a low-rank decomposition added to the frozen pretrained weight matrix [19]. Instead of updating a dense matrix directly, LoRA learns two much smaller matrices whose product forms a low-rank update. The method is appealing because it dramatically reduces the number of trainable parameters while preserving the original frozen model, and the resulting adapter can be merged into the base weights for inference if desired.

LoRA is not a single fixed recipe. Its behaviour depends especially on target modules and rank, which control where and how much adaptation capacity is added. In practice, much of the literature uses LoRA as a pragmatic fine-tuning tool rather than as the main object of study, so there is less controlled evidence on its behaviour in domain-adaptive continued pretraining than in downstream benchmark fine-tuning.

1.3.4 Quantisation and QLoRA

Quantisation aims to reduce memory use by representing model parameters in fewer bits. Dettmers et al. show that transformer inference can be made substantially more memory-efficient with 8-bit matrix multiplication while maintaining strong performance by handling outlier features explicitly [8]. This broader line of work establishes an important idea for large language models: lower precision is not only a deployment trick, but also a modelling choice that can change the feasibility frontier of large-model use.

QLoRA extends that idea into fine-tuning by storing the frozen base model in 4-bit precision

while backpropagating through it into LoRA adapters [9]. Its main innovations include the NF4 data type, double quantisation, and paged optimisers. In the literature, QLoRA is primarily presented as a method for making large-model fine-tuning feasible at lower memory cost, especially in instruction-tuning settings. That contribution is significant, but it does not by itself settle the question pursued here.

Quantising the frozen base model changes the numerical representation through which adaptation is computed, so it can affect activations, gradient signals, convergence behaviour, and the balance between memory savings and quality [8, 4]. The dissertation therefore focuses not on whether QLoRA can work at all, but on how the quantised and unquantised PEFT paths compare when data and optimisation opportunity are matched.

1.4 Evaluation Lenses and Legal-Domain Context

1.4.1 Quality, Efficiency, and Stability as Joint Evaluation Axes

Much of the existing literature foregrounds only part of the trade-off space: adaptive-pretraining studies often emphasise downstream performance gains [13, 5], LoRA foregrounds parameter efficiency relative to full fine-tuning [19], and QLoRA foregrounds memory reduction and large-model accessibility [9]. For a controlled comparison, however, held-out quality, memory, runtime, and stability must be treated jointly rather than as separate or selectively reported outcomes.

1.4.2 Legal NLP and UK Employment-Law Text

Legal NLP has grown rapidly, but much of the public literature focuses on broad benchmarks, large heterogeneous legal corpora, or downstream predictive tasks. LEGAL-BERT studies domain adaptation within the legal domain [5]; LexGLUE provides a benchmark suite for legal language understanding in English [6]; and Pile of Law illustrates the scale and heterogeneity of open legal pretraining data [14]. More recent UK-focused corpus work, such as the Cambridge Law Corpus, reinforces the point that even within a single national jurisdiction legal text remains broad and heterogeneous [28]. Together, these works show both that legal text is a viable domain for specialised language modelling and that “legal text” is far from homogeneous.

That heterogeneity is precisely why a narrower subdomain is useful here. For the present study, the curated corpus was designed to represent a narrower practical legal subdomain than a broad all-jurisdiction legal corpus. Tribunal judgments repeatedly address recognisable workplace disputes and legal tests: unfair dismissal, redundancy, discrimination, reasonable adjustments, wages, employment status, procedural fairness, and remedies. Layer B then adds expository material on the same world of concepts, including ACAS guidance on dismissals, disciplinary procedures, flexible working, TUPE, redundancy, and reasonable adjustments. This makes the corpus suitable for studying adaptation dynamics, because improvements in held-out perplexity can be interpreted more plausibly as domain-adaptive effects within the assembled corpus rather than as learning across a diffuse mix of unrelated legal subgenres.

At the same time, existing legal NLP work tends to emphasise classification, retrieval, summarisation, or judgment prediction rather than the intrinsic behaviour of continued pretraining itself. The present dissertation therefore does not treat legal text merely as an application area. It uses a legal-domain corpus to investigate a methodological question about quantised parameter-efficient adaptation under controlled data and training settings.

1.4.3 Efficiency and Optimisation Trade-offs in Prior Work

Prior work suggests that memory savings, runtime behaviour, and adaptation quality do not necessarily move together. LoRA reduces trainable parameter count substantially relative to full fine-tuning [19], while QLoRA makes larger base models trainable at much lower memory cost [9]. This review did not identify much tightly controlled evidence on how those methods compare in intrinsic domain-adaptive continued pretraining when the number of optimiser steps, the data, and the runtime policy are fixed. Earlier legal NLP work offers little help here, because its dominant emphasis is on downstream task performance rather than on jointly analysing memory, runtime, and stability.

1.5 Research Gap and Project Positioning

1.5.1 Research Gap and Project Positioning

Table 1.1 shows that the most relevant studies typically isolate only part of the present problem: adaptive pretraining without PEFT comparison, PEFT methods without intrinsic domain-adaptive evaluation, or legal-domain resources and benchmarks without matched LoRA-versus-QLoRA comparison. Among the papers reviewed here, none combines causal-LLM continued pretraining, a narrow legal subdomain, matched LoRA versus QLoRA exposure, and joint analysis of intrinsic quality, memory, runtime, and stability.

Study	Main contribution	Model / adaptation regime	Evaluation focus	Missing relative to this dissertation
Gururangan et al. (2020)	General motivation for domain-adaptive pretraining	Adaptive pretraining of pretrained language models	Downstream task performance	Not PEFT, not quantised PEFT, not causal-LLM LoRA/QLoRA comparison
Chalkidis et al. (2020)	Encoder-style legal-domain pretraining/adaptation	Legal-domain encoder adaptation	Downstream legal tasks	Not causal-LLM continued pretraining, not LoRA/QLoRA, not matched intrinsic comparison
Xie et al. (2024)	Efficient continual pretraining for domain-specific LLMs	Causal-LLM continual pretraining	Domain-specific task performance and continual-pretraining efficiency	Not PEFT comparison, not legal subdomain, not matched LoRA/QLoRA design
Hu et al. (2021)	Foundational LoRA method	Parameter-efficient adaptation via low-rank updates	Fine-tuning efficiency and task performance	Not domain-adaptive continued pretraining, not quantised comparison, not legal-domain study
Dettmers et al. (2023)	Foundational QLoRA method	Quantised PEFT for large language models	Feasibility and task fine-tuning performance	Not matched intrinsic LoRA/QLoRA DAPT comparison in law
Chalkidis et al. (2022)	Legal benchmark suite	Legal NLP task benchmarking	Downstream benchmark performance	Not continued pretraining, not PEFT comparison, not intrinsic evaluation
Henderson et al. (2022)	Responsible legal-data filtering and large legal corpus	Large legal and administrative text resource	Corpus construction and filtering	Not narrow subdomain adaptation, not LoRA/QLoRA, not controlled intrinsic study
Östling et al. (2023)	UK court-case corpus with metadata and benchmark annotations	UK legal text resource	Corpus construction and benchmarking context	Not employment-law-specific adaptation, not PEFT comparison, not intrinsic quantisation study

Table 1.1: Positioning of the most directly relevant studies reviewed in Chapter 1.

Within the literature reviewed here, three linked gaps remain. First, while adaptive pretraining and legal-domain pretraining are well motivated [13, 5, 31], there is comparatively limited controlled evidence on parameter-efficient domain-adaptive continued pretraining in causal large language models. Second, although LoRA and QLoRA are widely used [19, 9], much of the available evidence concerns task fine-tuning or instruction tuning rather than matched intrinsic comparison. Third, legal NLP provides broad corpora and task benchmarks [6, 14, 28], but less direct evidence on quantisation effects within a narrow legal subdomain under fixed resource constraints. This dissertation addresses those gaps through a controlled intrinsic study around a frozen UK employment-law corpus, a matched LoRA versus QLoRA protocol, and a multi-objective evaluation spanning held-out perplexity, memory, runtime, and stability.

Chapter 2

Methodology

This chapter defines the methodological design of the dissertation and the practical choices used to turn that design into a controlled comparative study. It explains the study design and protocol governance, then describes the experimental conditions, corpus construction, training procedure, and evaluation plan used for the final intrinsic comparison.

2.1 Study Design and Protocol Governance

2.1.1 Comparative Study Design

The dissertation is designed as a controlled comparative experiment in domain-adaptive continued pretraining on a fixed corpus of UK employment-law text. The primary independent variable is the adaptation method: standard LoRA with an unquantised **bf16** base model, versus QLoRA with a frozen 4-bit NF4 base model and **bf16** compute. A secondary independent variable is adapter rank within the quantised setting, used to test whether additional adapter capacity materially narrows any quality gap observed between LoRA and QLoRA. The dependent variables are grouped into four evaluation axes: held-out intrinsic quality, memory use, runtime efficiency, and training stability.

The study is deliberately framed around matched optimisation exposure rather than condition-specific best-case performance. The aim is to isolate the effect of 4-bit quantisation under shared data and training settings, not to maximise each method through separate tuning. For that reason, the corpus, split, sequence length, optimiser family, evaluation cadence, runtime policy, and seed set were held fixed across the final comparison. This makes any observed difference in held-out perplexity, memory, runtime, or stability more plausibly attributable to the quantised versus unquantised adaptation path rather than to unequal data exposure or a broader hyperparameter search.

Fairness is defined here as equality of experimental opportunity. Each condition sees the same corpus, train-validation partition, sequence length, effective batch size, learning-rate schedule, optimiser-step count, checkpoint-selection rule, and training seeds. QLoRA necessarily changes how the frozen base model is represented and executed, but separate condition-specific retuning would answer a different question from this controlled comparison.

2.1.2 Protocol Evolution and Final Freeze

The final reportable method is governed by `protocol_v3`, which is the authoritative frozen intrinsic protocol for the dissertation. Earlier protocol versions, local calibration runs, and target-hardware smoke tests were retained for feasibility validation, instrumentation checking, and the freezing of constants, but were not treated as substitutes for the final multi-seed

comparative evidence.

The protocol evolved through corpus stabilisation, pipeline validation, and target-hardware budget freezing. The key final change was the reduction of the intrinsic step budget from 750 to 350 after the first matched LoRA/QLoRA phase-1 pair under the earlier `protocol_v2` settings reached best overall validation perplexity at step 350 before degrading under further training. This freeze should be read as a pragmatic target-hardware decision rather than as a universal optimum. Short 4070 runs were useful for end-to-end validation and feasibility checks, but the final budget was frozen from matched L40S pilots because those share the execution context of the reportable campaign.

After `protocol_v3` was frozen, no changes were permitted to the corpus contents, split policy, sequence length, adapter targets, optimiser settings, runtime policy, step budget, seed set, or run matrix. The only permitted reruns were exact reruns of an already-frozen configuration following infrastructure failures such as cluster launch issues, shared-cache contention, or incomplete artifact generation. This governance model was used to prevent informal recipe drift and to preserve a clear distinction between scientific changes and operational fixes.

2.2 Experimental Conditions and Model Configuration

2.2.1 Run Matrix and Fairness Criteria

The final intrinsic run matrix contains four experimental conditions and three training seeds per condition, giving twelve reportable training runs in total. Phase 1 is the core matched comparison between LoRA and QLoRA at the same adapter rank. Phase 2 is a rank sweep within QLoRA, intended to test whether additional adapter capacity recovers some of the quality potentially lost to quantisation. The split seed remains fixed at 42 for all conditions so that between-run variation reflects optimisation variance rather than repeated data resampling.

Phase	Condition	Rank	Alpha	Training seeds
Phase 1	LoRA	16	32	42, 43, 44
Phase 1	QLoRA	16	32	42, 43, 44
Phase 2	QLoRA	32	64	42, 43, 44
Phase 2	QLoRA	64	128	42, 43, 44

Table 2.1: Frozen intrinsic run matrix defined by `protocol_v3`.

Role	Parameter	Protocol setting
Varied	Adaptation path	LoRA on an unquantised bf16 base versus QLoRA on a 4-bit NF4 base
Varied	Adapter rank	r=16 for Phase 1; r=16,32,64 within the QLoRA rank sweep
Fixed	Data and split	Same corpus manifest, document-level split, and split seed 42
Fixed	Model and representation	Same Llama-3-8B base family, tokenizer, 1024-token chunks, and packed validation views
Fixed	Optimisation budget	Same effective batch size of eight sequences, learning rate, scheduler, 350 optimiser steps, and 50-step evaluation cadence
Fixed	Adapter surface	Same attention targets q_proj , k_proj , v_proj , o_proj and dropout 0.05
Fixed	Runtime and replication	Same L40S runtime policy, sdpa attention path, and training seeds 42, 43, and 44

Table 2.2: Fixed and varied parameters in the final `protocol_v3` comparison.

Table 2.2 summarises how the fairness definition is operationalised. The principal varied factor is the quantisation state of the frozen base model, with QLoRA adapter rank treated as a secondary design variable.

For the core Phase 1 comparison, the primary quality endpoint is best-checkpoint overall validation perplexity under the frozen checkpoint-selection rule. Baseline perplexity, final-checkpoint perplexity, and layer-stratified perplexities are retained as supporting evidence rather than alternative primary selection criteria.

2.2.2 Base Model and Tokenizer

All final conditions use `meta-llama/Meta-Llama-3-8B` as the base model. This choice reflects a compromise between scientific relevance and practical feasibility. An 8B-parameter base model is large enough for the LoRA versus QLoRA trade-off to be meaningful under constrained hardware, but still feasible to run on the target L40S platform without resorting to further architectural changes such as shorter context lengths or reduced batch size. The project uses the base model rather than an instruction-tuned variant because the objective is domain-adaptive continued pretraining measured by language-modelling behaviour, not instruction-following quality [12].

Tokenisation uses the Llama-3 tokenizer with a vocabulary size of 128,256. The maximum sequence length is fixed at 1024 tokens across all conditions. In the implementation, if the tokenizer has no explicit padding token, the end-of-sequence token is reused as the padding token so that fixed-length batches can be constructed consistently. The sequence length is treated as a frozen scientific constant rather than a tuning variable because changing it would alter both the optimisation budget and the effective modelling context, thereby confounding the comparison.

2.2.3 Adapter Configurations

The LoRA configuration used for the core comparison applies adapters only to the attention projection matrices `q_proj`, `k_proj`, `v_proj`, and `o_proj`, with rank 16, scaling factor `alpha=32`, and dropout 0.05. The base model remains frozen and only the low-rank adapter parameters are updated during training [19, 22]. Attention-only adapters were chosen to keep the adaptation surface compact and interpretable. Under the fixed Llama-3-8B attention

dimensions used here, this corresponds to 13,631,488 trainable parameters for $r=16$ in both LoRA and QLoRA, 27,262,976 for $r=32$, and 54,525,952 for $r=64$. Relative to an 8B-parameter base model, these are approximately 0.17%, 0.34%, and 0.68% of base-model size, respectively. This makes the core Phase 1 comparison easier to interpret: LoRA $r=16$ and QLoRA $r=16$ differ in base-weight precision, not in trainable adapter footprint, while the QLoRA rank sweep increases capacity in a controlled way. Broadening the target-module set would have introduced a larger design-space change than is needed to study the effect of quantisation under matched budgets. In particular, adding MLP targets would materially increase trainable parameter count and adaptation capacity at the same time as changing where the adapters act, thereby making any later quality difference harder to attribute cleanly.

The QLoRA conditions retain the same adapter targets and dropout, but quantise the frozen base model to 4-bit NF4 with double quantisation enabled and `bf16` used as the compute dtype. The rank-sweep conditions increase adapter capacity to $r=32$, `alpha=64` and $r=64$, `alpha=128`, respectively. Rank is therefore the only deliberate capacity change explored in the fixed `protocol_v3` settings. This keeps the study narrow enough for clear interpretation: if higher-rank QLoRA improves held-out modelling quality, that improvement can be discussed specifically as a capacity effect within the quantised regime rather than as a consequence of broader architectural changes [9, 23].

2.3 Corpus Construction and Data Preparation

2.3.1 Corpus Design and Source Selection

The training corpus was designed specifically for domain-adaptive continued pretraining on UK employment law rather than for supervised downstream classification. It therefore consists of raw domain text organised into two layers with distinct but complementary roles. Layer A contains employment tribunal judgments and provides the main adjudicative signal: factual narratives, procedural framing, statutory language, and judicial reasoning. Layer B contains doctrinal and guidance material drawn from ACAS guidance and codes of practice, GOV.UK employment and working guidance, and House of Commons Library briefings [2, 10, 11, 18]. The manually selected source set included exemplar pages and briefings such as [3, 1, 17, 7]. Layer B is smaller, but provides cleaner expository signal and practical explanations of statutory concepts.

This layered design serves two purposes. First, it narrows the study to a curated practical subdomain rather than treating “legal text” as a single, uniform category. The assembled corpus was intentionally constructed so that held-out intrinsic differences could be interpreted against recurrent employment-law terminology, fact patterns, forum conventions, and source hierarchies rather than against a diffuse mix of unrelated legal genres. Second, it allows the corpus to balance primary adjudicative text with high-signal explanatory material. Tribunal judgments dominate the token budget, but guidance and doctrine reduce the risk that the corpus becomes only a collection of case-specific narratives without broader doctrinal vocabulary.

2.3.2 Data Collection and Preprocessing

Layer A was collected from UK employment tribunal decision PDFs [15]. The extraction pipeline was refined iteratively to deal with heterogeneous document layouts and PDF-specific artefacts. The final version performs text extraction, boilerplate removal, whitespace normalisation, and paragraph reflow while preserving headings and numbering. Implementation detail for these stages is discussed in Chapter 3.

Quality filtering for Layer A was intentionally strict. A tribunal text was kept only if it contained at least 750 Llama-3 tokens and exhibited paragraph numbering, which was used as a proxy for substantive judicial reasoning. This filtering removed very short orders, extraction failures, and boilerplate-only documents without requiring task-specific labels. Importantly, the same cleaned texts were then reused across every experimental condition, so any residual preprocessing noise is symmetric with respect to the LoRA versus QLoRA comparison.

The threshold was moved from word space to token space so that corpus inclusion is controlled in the same representation as the later training budget.

Layer B followed a different pipeline because the source material is more heterogeneous in format but generally cleaner than tribunal PDFs. Authoritative guidance and doctrine sources were selected manually, then cleaned using a strictly subtractive pass that removed navigation, duplicated boilerplate, and link artefacts without paraphrasing legal content.

2.3.3 Tokenisation, Document Capping, and Corpus Freeze

All corpus sizing decisions were made in the Llama-3 token space because the training budget is defined in tokens. The post-reflow tribunal candidate set contained 271 documents and 3,105,775 tokens before capping, with 17 very long judgments contributing roughly one quarter of all Layer A tokens. To stop that small tail dominating the fixed budget, each judgment was capped at 20,000 tokens after reflow, truncating at the nearest preceding paragraph boundary. The cap affected only those 17 documents and reduced tribunal tokens to 2,497,335. Segmented sampling was rejected because it would have introduced artificial within-document discontinuities in a continued-pretraining setting.

After capping, Layer A documents were selected deterministically using a seeded greedy fill process to meet a target total corpus budget of approximately 1.8 million tokens while preserving the already-completed Layer B set. The final frozen corpus contains 170 Layer A judgments and 60 Layer B guidance or doctrinal documents, for a total of 230 files and 1,802,427 tokens. The final manifest was written to `data/domain_corpus/corpus_manifest.json` and then treated as immutable for the reportable runs.

2.3.4 Train–Validation Split and Leakage Control

Intrinsic evaluation uses a document-level 85/15 train–validation split. The split is stratified by layer and by token-length quartile within each layer, using a fixed split seed of 42. Stratification by layer preserves the overall corpus composition in the held-out set, while stratification by token quartile reduces the risk that validation quality is dominated by a disproportionate

number of unusually short or unusually long documents. In the development log, this quartile scheme was adopted specifically to stabilise held-out perplexity estimates and to reduce short-document skew in the validation partition.

Document-level splitting was chosen to prevent leakage between training and validation. Because the training representation concatenates complete documents within each split before chunking them into fixed-length sequences, splitting at the chunk level would have allowed adjacent or overlapping material from the same source document to appear in both training and validation. In a legal corpus, where case-specific names, facts, and phrasing may recur heavily within a single judgment, such leakage would make held-out perplexity artificially optimistic.

Layer	Train docs	Val docs	Total docs	Train tokens	Val tokens	Total tokens
Layer A	144	26	170	1,362,559	239,631	1,602,190
Layer B	51	9	60	177,970	22,267	200,237
Overall	195	35	230	1,540,529	261,898	1,802,427

Table 2.3: Frozen document-level train-validation split used for the intrinsic study.

No separate intrinsic test split is used in the final pipeline. The study prioritises a fixed document-level validation set for controlled multi-seed comparison under the available data and hardware budget, and therefore reports held-out validation perplexity rather than independent test-set generalisation. This scope is revisited in Chapter 4.

2.3.5 Training Representation

The training representation follows the actual behaviour of the implemented pipeline rather than a more idealised document model. Documents within each split are first tokenised with special tokens enabled and without truncation. Within each split, documents are ordered deterministically by relative path before concatenation so that the packed token stream is stable across reruns. The resulting token sequences are then concatenated within the split and cut into contiguous 1024-token chunks. Any remainder shorter than 1024 tokens is dropped to avoid ragged batches. This means that the optimisation budget is expressed directly in full fixed-length sequences rather than in partially padded documents [25].

This representation is implemented through deterministic Python-level concatenation rather than map-batch grouping inside a dataset transform. That choice was made to avoid hidden token loss at batch boundaries during preprocessing. Although subtle, it matters methodologically: if different packing behaviour silently changes the number of effective training chunks, budget matching between conditions becomes less transparent.

The trade-off is that document boundaries are not preserved inside training chunks. Some sequences therefore span from one source document into the next, rather than representing intact judicial or doctrinal passages. This is a standard packing strategy for causal language modelling and is useful for packing efficiency and transparent budget accounting, but it reduces discourse-level structural fidelity. Held-out perplexity in this study should therefore be interpreted as perplexity under the packed fixed-length sequence representation, not as a direct measure of whole-document coherence [21, 25].

Using the frozen corpus and split, this representation yields 1,504 training chunks and 255

validation chunks overall. Separate validation views are also constructed for Layer A and Layer B so that the same checkpoint can be reported on overall held-out perplexity and on layer-specific held-out perplexity. These layer-specific views are used for analysis and interpretation, but not for checkpoint selection.

2.4 Training Environment and Procedure

2.4.1 Hardware and Software Environment

The final intrinsic campaign targets NVIDIA L40S hardware. Exact software versions are captured automatically for each run in a structured `environment.json` snapshot rather than being maintained manually in the report. Because memory use and runtime are reported outcomes, the execution environment forms part of the methodological record rather than mere operational background.

The runtime path uses scaled dot-product attention (`sdpa`) with `bf16` compute, while the quantised path additionally relies on `bitsandbytes` 4-bit loading with `NF4` and double quantisation [23, 20]. For the final report, the primary preserved evidence is the structured artifact pack produced by each run rather than raw stdout or stderr traces. Operational hardening of the launch path, such as cache isolation and wrapper fixes, is treated as an implementation matter in Chapter 3 because it did not alter any frozen scientific constant.

2.4.2 Training Budget and Runtime Policy

All final runs use the AdamW optimiser family with a learning rate of `2e-4`, cosine learning-rate decay, warmup ratio 0.05, and weight decay 0.01. The implementation relies on Hugging Face `TrainingArguments`, so optimiser beta, epsilon, and gradient-clipping behaviour follow the standard AdamW defaults used by that framework, consistent with the values frozen in `protocol_v3` [24]. The per-device training batch size is 2, the per-device evaluation batch size is 1, and gradient accumulation is fixed at 4, giving an effective batch size of eight sequences per optimiser step.

The study is step-budgeted rather than epoch-driven. Each run is limited to 350 optimiser steps, with evaluation and checkpoint saving every 50 steps and training logs every 10 steps. At 1024 tokens per sequence and eight sequences per optimiser step, this corresponds to 2,867,200 processed tokens, or about 1.86 epoch-equivalents of the frozen training set. Step budgeting preserves exact comparability of optimiser exposure across conditions even when the dataset size does not divide cleanly into whole epochs.

The runtime policy is also frozen across conditions. Gradient checkpointing is disabled for the final L40S protocol because target-hardware validation showed that it was not needed for feasibility, and the attention backend remains `sdpa`. The implementation also requires evaluation and save cadence to remain aligned and disables `model.config.use_cache` during training to avoid unnecessary memory overhead.

2.4.3 Baseline Evaluation and Checkpoint Selection

Every run begins with a baseline evaluation before any training takes place. The baseline records the held-out intrinsic quality of the starting model under the same evaluation frame later used for trained checkpoints, and it captures the pre-adaptation effect of quantisation itself. For LoRA conditions, the baseline is evaluated on the unquantised **bf16** base model. For QLoRA conditions, the baseline is evaluated on the 4-bit **NF4** quantised base model.

The QLoRA baseline uses a manual evaluation route because the **Trainer**-centred adapter path does not support the purely quantised base state directly. The metric target remains matched: the same held-out datasets, batch size, data collator, loss-to-perplexity conversion, and VRAM accounting logic are used. After baseline evaluation, adapted checkpoints from both methods are assessed under the shared scheduled evaluation procedure used for checkpoint comparison and selection.

Checkpoint selection is based on minimum overall validation perplexity (`eval_perplexity`). Layer-specific validation perplexities for Layer A and Layer B are recorded at every evaluation point, but they do not determine checkpoint choice. This rule keeps the primary selection criterion aligned with the overall intrinsic study objective while still exposing potentially different adaptation behaviour across the adjudicative and doctrinal layers. Both the best checkpoint and the final checkpoint are retained. This is important because the earlier matched target-hardware pilots showed that best and final checkpoints can differ materially under a fixed step budget.

2.5 Evaluation, Aggregation, and Validity

2.5.1 Outcome Measures

The primary quality outcomes are held-out validation perplexities measured before training, at each evaluation checkpoint, and at the end of training. These are reported for the overall validation set and separately for Layer A and Layer B. For each run, the dissertation therefore distinguishes between baseline, best-checkpoint, and final-checkpoint intrinsic quality rather than reporting only a single end-of-run score.

As argued in Chapter 1, perplexity is used here because it aligns directly with the causal language-modelling objective optimised during domain-adaptive continued pretraining. It is not treated as a complete measure of legal capability. The methodology therefore uses held-out perplexity to measure intrinsic adaptation quality, while leaving downstream task performance outside the frozen scope of the core comparison.

Memory is reported through baseline-evaluation, training, evaluation-only, and overall run-level VRAM peaks. Runtime is reported through whole-run training time and step-time summaries, including mean, median, 95th percentile, standard deviation, and sample count.

Stability is reported as objective failure or non-finite events rather than as a subjective impression of whether a run “looked stable”. The structured summary records non-finite losses or gradient norms, out-of-memory events, early termination, and related run-completion metadata. Softer dynamics such as noisier loss movement remain interpretive rather than part of the formal

counted stability definition; the implementation of this logging is described in Chapter 3.

2.5.2 Aggregation and Analysis Plan

The final intrinsic conditions are repeated across the fixed training seeds 42, 43, and 44. Within each run, checkpoint selection is performed independently using the minimum overall validation perplexity rule described above. Across runs, saved artefacts are aggregated into seed-level tables, condition-level summaries, and merged evaluation curves. For each condition, the core summary statistics are the mean, standard deviation, minimum, maximum, and sample count over the completed seeds.

Because each reportable condition is repeated only three times, the planned cross-seed analysis is primarily descriptive rather than inferential. The dissertation therefore reports seed-level values, condition-level summaries, and seed-aligned paired differences for matched conditions, but does not treat the resulting evidence as a high-power basis for formal hypothesis testing. Taken together with the fixed train-validation split and the pilot-informed shared step budget, the final evidence regime should be read as controlled comparative evidence under the fixed `protocol_v3` settings rather than as exhaustive schedule-optimised or test-set-based generalisation evidence.

Interpretation is explicitly multi-objective, considering intrinsic quality, memory demand, and runtime cost jointly rather than ranking conditions by perplexity alone. Per-run inspection remains useful for interpreting learning curves, checkpoint locations, or seed-specific anomalies, but condition-level comparison is the basis for the dissertation’s main claims.

2.5.3 Reproducibility, Validity, and Limitations

Auditability and practical repeatability are supported by frozen manifests, versioned split files, explicit YAML configurations, and per-run structured artifacts. The repository history records the protocol documents, launch scripts, code changes, and calibration materials that produced those artefacts; the detailed mechanics are discussed in Chapter 3. Seeds and ordering are fixed, but full kernel-level determinism is not enforced across the CUDA stack, so some residual low-level variance may remain despite seed control [29].

Internal validity is strengthened by the matched data, optimisation, and runtime settings. The main remaining threats are inherent to the comparison itself, such as quantisation-specific kernel behaviour or hardware-level runtime effects. Those effects are treated as measured systems outcomes alongside intrinsic quality.

External validity is necessarily limited. The study focuses on a single base model family, a single hardware platform, a single legal subdomain, and a single intrinsic evaluation regime. The results will therefore support claims about controlled domain-adaptive continued pretraining under these conditions, not about all forms of legal adaptation or all quantised fine-tuning settings. Further limitations arise from the finite corpus size, residual preprocessing noise, and the absence of a separate intrinsic test split.

Chapter 3

Implementation and Validation

This chapter explains how the methodological design from Chapter 2 was realised in software and how the resulting pipeline was validated before the final reportable experiments were launched. Against the project requirements, the implementation had to deliver more than a working training script. It had to produce a frozen corpus, a stable split-generation path, a configuration-driven LoRA/QLoRA execution pipeline, structured per-run reports, and an aggregation route from individual runs to the summary tables and figures used in Chapter 4. The chapter therefore focuses on implementation structure, engineering challenges, and the validation steps that established confidence in the final artefacts.

3.1 Implementation Architecture

3.1.1 Pipeline Decomposition and Deliverables

The project was implemented as a staged pipeline. The first stage acquired and cleaned raw domain documents, producing reflowed tribunal text, cleaned guidance and doctrine text, token statistics, and a frozen corpus manifest. The second stage generated the document-level train-validation split and the packed training representation used by all reportable runs. The third stage executed baseline-first evaluation, adapter training, scheduled checkpoint evaluation, and final adapter export for each reportable condition. The fourth stage aggregated per-run reports into condition summaries and merged histories for visualisation and analysis.

Each stage wrote persistent outputs consumed by the next stage: the corpus manifest, split assignment, resolved run configuration, environment snapshot, dataset and budget summaries, checkpoint metrics, comparison-ready rows, and merged cross-run histories. This made the Chapter 4 results traceable from source documents through to condition-level aggregates.

3.1.2 Configuration-Driven Execution

Execution was configuration-driven throughout. Versioned experiment configurations specified the model family, adapter regime, hardware policy, paths, training budget, and reporting settings for each run. The same execution path was used for local smoke tests, target-hardware pilots, and the frozen `protocol_v3` matrix. Moving from exploratory validation to final comparison therefore required a configuration change, not a separate training program or informal shell-level adjustment.

Every completed run emitted a resolved record of the settings under which it had actually executed, together with a run manifest describing the resulting artefacts and completion status. A representative reportable run records its condition identity and seed, data basis, planned and completed step budget, logging schema version, and generated reports. The final comparative

evidence was therefore recoverable from saved state rather than reconstructed manually after the fact.

3.2 Data, Training, and Reporting Realisation

3.2.1 Corpus Preparation and Training-Data Packaging

The data pipeline implemented the corpus design from Chapter 2 in two distinct streams. For Layer A, the implementation had to handle noisy PDF layouts from tribunal judgments. The resulting pipeline performed text extraction, boilerplate and administrative-text removal, whitespace normalisation, and a dedicated reflow stage that reduced layout artefacts while preserving numbered paragraphs and other legal structure. Filtering then applied the two quality gates used later in the methodology chapter: a minimum token threshold in the Llama-3 token space and the requirement for paragraph numbering. These choices converted a heterogeneous source collection into a more consistent body of adjudicative text without introducing task labels or manual rewriting.

Layer B required a different implementation style. The source material was cleaner than tribunal PDFs but more heterogeneous in provenance and formatting, so automated extraction was followed by a strictly subtractive cleaning pass. The objective was not to rewrite or simplify the text, but to remove navigation, duplicated boilerplate, and link artefacts while leaving the legal content unchanged. The output of both streams then fed into token counting, document capping, and deterministic corpus selection. The post-cap statistics confirmed that the 20,000-token cap affected only a small right tail of very long judgments, while the final frozen corpus preserved the intended Layer A / Layer B balance.

Split generation and packaging used the same deterministic record. The split generator stratified by layer and token-length quartile, emitted a versioned split object, and recorded metadata for later inspection of layer composition and quartile allocation. The data loader then sorted documents by relative path before concatenating each split into fixed 1024-token chunks and creating the overall, Layer A, and Layer B validation views. Reruns therefore reused the same packed representation, and the layer-specific analysis in Chapter 4 rested on stable validation partitions.

3.2.2 Training, Evaluation, and Reporting Pipeline

The training pipeline implemented both LoRA and QLoRA through a shared baseline-first lifecycle: model and tokenizer loading, adapter attachment, pre-training held-out evaluation, scheduled training and evaluation, best-checkpoint selection, and final adapter export. This shared path ensured that the main difference between conditions remained the intended scientific difference rather than a divergent implementation route. At the same time, the project had to accommodate one genuine implementation asymmetry. A purely quantised base model could not be evaluated through the same adapter-aware `Trainer` route as the unquantised LoRA baseline, so a manual stratified baseline path was added for QLoRA while keeping the same held-out datasets, batch size, collator behaviour, and loss-to-perplexity conversion.

The reporting path was implemented as part of the training lifecycle. Each completed run emitted a structured report pack covering configuration, environment, dataset composition, budget, evaluation history, final metrics, memory usage, timing summaries, stability summaries, and a comparison-ready summary row. One representative completed run records the frozen split seed, the overall train/validation token counts, 1504 training chunks, the planned and completed 350-step budget, windowed step-time statistics, and final stability status. The Chapter 4 tables and figures are produced from these saved reports rather than assembled manually.

3.2.3 Orchestration and Failure Handling

Cluster execution was implemented as part of the project solution, not as an external operational detail. Launch wrappers performed preflight checks, environment bootstrapping, output-directory creation, and configuration dispatch, while keeping the frozen experiment outputs separate from transient cache state. This mattered because the reportable run matrix was large enough that execution hygiene directly affected whether the final artefacts would be complete and comparable.

The implementation also had to deal with failure paths professionally. OOM events and non-OOM exceptions were instrumented so that failed runs still wrote manifests and stability summaries rather than disappearing into a terminal traceback. Later cluster issues, such as shared-cache contention and wrapper stdout failures, were treated as engineering faults in the launch path rather than as reasons to reopen the scientific protocol. This distinction was crucial: it preserved the frozen experimental design while still allowing the execution environment to be hardened enough for the final campaign to complete reliably.

3.3 Validation and Verification

3.3.1 Validation Strategy

Validation was treated as a staged process with three aims: to verify that the data pipeline produced the intended corpus and split artefacts, to verify that the runtime path behaved correctly for baseline and training, and to verify that the reporting path emitted complete and internally consistent artefacts. This strategy matters for the implementation criterion because the final dissertation claims rely on more than numerical outputs alone. They also rely on confidence that those numbers were generated under the intended protocol and captured in a form that could be inspected and reproduced.

Accordingly, earlier experiments are best understood as validation evidence rather than as part of the final comparative sample. Local and target-hardware pilots were used to establish feasibility boundaries, exercise the baseline and checkpoint logic, and check the integrity of the artefact schema before the final matrix was queued. This kept the final results chapter focused on the frozen evidence regime while still allowing the dissertation to demonstrate that the underlying pipeline had been tested thoroughly enough to trust.

Component	Main risk	Check performed	Artefact evidence	Result
Corpus cleaning and selection	PDF layout noise or low-signal documents	Reflow and token-space filtering in <code>reflow_tribunal_text.py</code> and <code>select_corpus.py</code>	<code>corpus_manifest.json</code> ; <code>corpus_final/</code>	Frozen two-layer corpus with strict Layer A quality gates
Split and packing	Leakage or unstable validation partitions	Layer/quartile stratification, duplicate- <code>reldpath</code> guard, deterministic relative-path ordering	<code>intrinsic_splits.json</code> ; per-run <code>dataset_summary.json</code>	Stable train/validation split and 1504/255 packed chunks
Baseline and training lifecycle	LoRA/QLoRA metric mismatch	Shared lifecycle in <code>train_lora.py</code> ; manual QLoRA baseline route checked against same collator and loss conversion	<code>baseline_eval_metrics.json</code> ; <code>eval_summary.csv</code> ; <code>final_metrics.json</code>	Baseline-first and 50-step evaluations available for every reportable run
Failure and stability handling	Silent OOM or non-finite failures	Stability callbacks, exception handling, failed-run manifest writing	<code>stability_summary.json</code> ; <code>stability_events.jsonl</code> ; <code>run_manifest.json</code>	Failures became auditable events; final matrix completed without counted failures
Environment and configuration	Irrecoverable runtime settings	Resolved configuration and software/environment snapshot emitted at run start	<code>resolved_config.yaml</code> ; <code>environment.json</code>	Run settings are recoverable from each report pack
Aggregation and plotting	Manual transcription error	<code>compare_runs.py</code> merged <code>comparison_row.json</code> files; figure script consumed aggregate CSVs	<code>reports_index/protocol_v3/*.csv</code> ; Chapter 4 figure/table pack	Twelve seed-level runs reduced to deterministic condition summaries

Table 3.1: Validation matrix for the implemented experimental pipeline.

3.3.2 Smoke Tests, Feasibility Checks, and Pilot Evidence

Two layers of smoke testing were especially important. Earlier local consumer-GPU runs established practical feasibility boundaries and validated the newer instrumentation around baseline-first evaluation, cache clearing, OOM capture, and best-checkpoint enforcement. These checks showed that sequence length 1024 remained a meaningful stress point on a 12GB-class device and that failure cases produced usable artefacts rather than silent corruption. They also helped separate engineering feasibility from the final scientific comparison: the project could demonstrate that the pipeline handled success and failure cleanly before moving to the target hardware.

Target-hardware pilots on the L40S platform then validated the exact runtime path used later for the final model family. In particular, the quantised baseline route required two debugging iterations before it became reliable: the first because the standard trainer route rejected a purely quantised baseline model, and the second because the manual baseline path initially hit a kernel availability failure. Resolving those issues before the full matrix was launched was important because it meant the final LoRA and QLoRA runs entered the frozen campaign with their baseline semantics already validated. The matched pilot pair also provided the evidence used later to freeze the 350-step budget: both conditions reached their best overall validation

perplexity at step 350 and degraded thereafter, so the shorter shared schedule was justified directly from target-hardware evidence.

3.3.3 Instrumentation Validation and Artefact Integrity

The reporting layer was validated alongside the training path. Memory tracking was checked across baseline, training, evaluation-only, and run-level summaries; timing summaries were checked against the windowed optimizer-step measurements emitted during training; and stability summaries were checked against recorded events and completion status. These checks support the multi-objective comparison in Chapter 4, where memory, timing, and stability are primary evidence rather than auxiliary logs.

Aggregation was validated in the same way. Each completed run emitted a comparison-ready summary row, and the merge process was checked to ensure that per-run outputs entered the cross-run tables and merged evaluation histories without manual transcription. The condition-level summaries and figures used in the results chapter are deterministic products of saved artefacts.

3.3.4 Engineering Challenges and Quality Controls

The main implementation challenge was making the whole pipeline robust enough for a controlled comparative study. Tribunal PDFs contained layout noise that had to be normalised without destroying numbered legal structure. The quantised baseline path required a special handling route that preserved metric comparability. Cluster execution introduced cache-concurrency and launcher-output problems that had to be solved without altering the scientific constants frozen in the protocol. Each issue affected comparability or artefact integrity, so each had to be resolved before the final campaign could be treated as reportable evidence.

Several engineering controls were adopted to improve output quality: configuration resolution, schema versioning, deterministic ordering, structured artefact emission, and failure-path manifests. Together with the staged validation process above, these controls left an evidence trail that links the implemented system to the comparative results in Chapter 4.

Chapter 4

Results, Evaluation and Discussion

This chapter presents the empirical findings of the dissertation under the matched data, optimisation, and reporting frame established in Chapter 2. The focus throughout is the completed `protocol_v3` run set, with earlier pilots used only where needed to bound interpretation.

4.1 Reporting Basis and Evaluation Frame

4.1.1 Completed Run Set and Basis of Analysis

The final analysis uses only the completed `protocol_v3` intrinsic runs: twelve reportable runs covering LoRA r16, QLoRA r16, QLoRA r32, and QLoRA r64, each repeated for seeds 42, 43, and 44. All twelve runs completed under the same logging schema, so no condition is provisional or excluded.

Earlier local smoke tests, target-hardware pilots, and implementation validation runs are used only to explain protocol design in Chapter 2. The comparative results below are reproduced from the saved per-run artifacts and aggregated summaries under `outputs/reports_index/protocol_v3`, not from manual notes. The evidence regime is therefore deliberately bounded: a fixed comparative run matrix rather than an open-ended pool of exploratory experiments.

4.1.2 Baseline and Checkpoint-Selection Frame

The primary quality endpoint is best-checkpoint overall validation perplexity under the checkpoint rule from Chapter 2. Baseline, final, and layer-specific perplexities support interpretation but are not competing primary endpoints. The baseline-first design matters because LoRA and QLoRA start from different numerical base states: the unquantised LoRA baseline averages 7.9612, whereas the three QLoRA conditions share a quantised baseline of 8.3170. This separates the pre-adaptation cost of quantisation from the amount recovered by adapter training.

For all four conditions, the best overall validation perplexity occurs at the final evaluated checkpoint at step 350, so best and final overall perplexities coincide. Cross-seed interpretation remains descriptive rather than inferential: with three repeated seeds per condition, the chapter uses means, sample standard deviations, seed-level values, and paired differences to judge direction and consistency, without claiming formal statistical significance.

Table 4.1 gathers the chapter’s main numerical comparisons in one place. It serves as the reference table for the condition means discussed in the quality and trade-off sections that follow.

Table 4.1: Master summary of the four completed `protocol_v3` conditions. Memory and timing values are condition means across the shared seed set.

Condition	n	Baseline overall	Best overall	SD	Final overall	Best L-A	Best L-B	Δ best overall	Train peak VRAM (GB)	Mean step time (s)	Train runtime (s)
LoRA r16	3	7.9612	7.2670	0.0018	7.2670	7.5725	4.5762	-0.6942	15.126	1.993	1088.7
QLoRA r16	3	8.3170	7.4125	0.0009	7.4125	7.7238	4.6748	-0.9045	7.437	4.958	2688.9
QLoRA r32	3	8.3170	7.3832	0.0013	7.3832	7.6961	4.6376	-0.9338	7.590	4.983	2703.2
QLoRA r64	3	8.3170	7.3837	0.0012	7.3837	7.7011	4.6158	-0.9332	7.894	4.988	2714.4

4.2 Intrinsic Quality Results

4.2.1 Primary LoRA versus QLoRA Comparison

The core Phase 1 comparison asks whether 4-bit quantisation changes intrinsic adaptation quality when LoRA and QLoRA are given the same corpus, the same optimiser exposure, the same checkpoint-selection rule, and the same training seeds. On the primary endpoint, the answer is yes. Averaged across the three matched seeds, **LoRA r16** achieves a best overall validation perplexity of 7.2670, whereas **QLoRA r16** reaches 7.4125. With the same corpus, document split, sequence length, optimiser settings, 350 optimiser steps, and shared seeds, the quantised condition therefore remains worse by 0.1454 perplexity on the held-out overall validation set.

The baseline-first design helps clarify where that gap comes from. LoRA begins from a better starting point (7.9612) than QLoRA (8.3170), so part of the final difference is already present before adaptation begins. At the same time, the quantised condition improves more relative to its own baseline: **QLoRA r16** reduces overall perplexity by 0.9045 from baseline to best checkpoint, whereas **LoRA r16** improves by 0.6942. Quantised adaptation therefore recovers much of its initial baseline penalty, but not enough to match the unquantised LoRA condition under the same controlled adaptation exposure.

Figure 4.1 makes this primary overall ordering visually explicit: all three quantised conditions lie above **LoRA r16** on the best-checkpoint overall perplexity axis, with **QLoRA r32** and **QLoRA r64** nearly tied and both clearly ahead of **QLoRA r16**.

This ordering is highly consistent across the shared seed set. The paired seed-aligned differences in best overall perplexity (**QLoRA r16** minus **LoRA r16**) are +0.1447, +0.1438, and +0.1478, far larger than the within-condition standard deviations (0.0018 and 0.0009). The direction is therefore stable across the matched seeds.

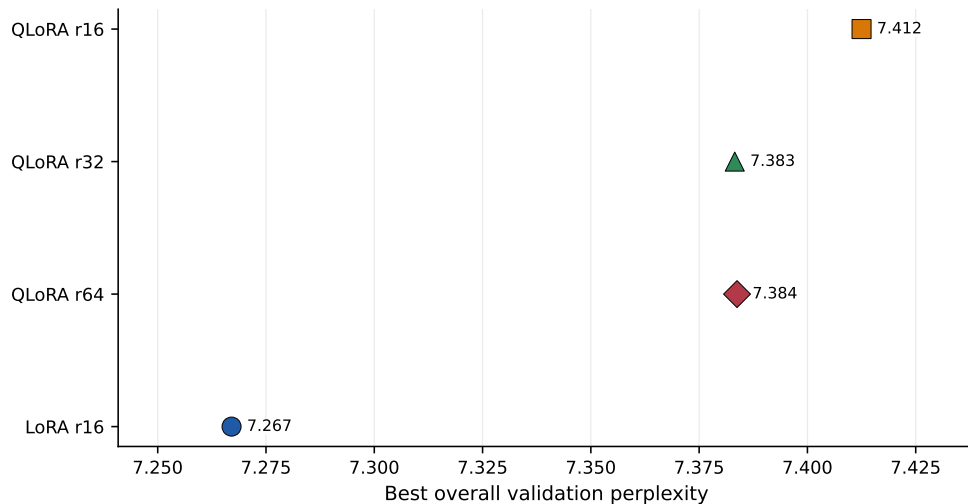


Figure 4.1: Best-checkpoint overall validation perplexity by completed condition. Conditions are ordered as the QLoRA rank sweep followed by the unquantised LoRA reference condition. Cross-seed variability is reported numerically in Table 4.1.

The same overall pattern is visible in the layer-specific validation views. For Layer A, LoRA **r16** reaches a mean best perplexity of 7.5725 versus 7.7238 for QLoRA **r16**, a gap of 0.1513. For Layer B, the corresponding means are 4.5762 and 4.6748, a smaller but still clear gap of 0.0986. The quantised condition is therefore worse on both validation strata, not only on the combined measure. The difference appears larger on the tribunal-dominated Layer A split than on Layer B, although the central claim is simpler: under matched optimisation budgets, LoRA **r16** delivers the best held-out intrinsic quality in the core LoRA–QLoRA comparison.

4.2.2 QLoRA Rank Sweep

The QLoRA rank sweep asks a narrower question than the primary Phase 1 comparison: once the frozen base model is quantised, does increasing adapter capacity materially recover intrinsic quality under the same fixed budget? Within the final run set, the answer is yes for the move from QLoRA **r16** to QLoRA **r32**, but much less clearly for the move from **r32** to **r64**.

On the primary endpoint, mean best overall validation perplexity falls from 7.4125 at **r16** to 7.3832 at **r32** and 7.3837 at **r64**. The seed-aligned paired differences are consistent for both higher-rank conditions: **r32** improves on **r16** by -0.0279 , -0.0317 , and -0.0282 , while **r64** improves by -0.0291 , -0.0284 , and -0.0287 . Increasing rank within QLoRA therefore recovers some of the intrinsic quality lost by QLoRA **r16**.

That recovery, however, does not close the primary LoRA–QLoRA gap. Even the best quantised condition remains above the LoRA **r16** mean of 7.2670 by approximately 0.1162 perplexity. The rank sweep therefore refines the dissertation’s central result rather than overturning it: additional adapter capacity helps within QLoRA, but under the same 350 optimiser steps and 8192-token optimiser-step exposure it does not fully restore the quality of the matched unquantised LoRA condition.

The difference between QLoRA **r32** and QLoRA **r64** is much smaller and less decisive. Their mean best overall perplexities differ by only 0.0005, and the paired seed-aligned differences are

not directionally uniform (-0.0012 , $+0.0033$, and -0.0005 for **r64** minus **r32**). On the overall primary endpoint, this is effectively a tie at the resolution of the present study. The layer-specific views help explain why. On Layer A, QLoRA **r32** is slightly better than QLoRA **r64** (7.6961 versus 7.7011), whereas on Layer B, QLoRA **r64** is clearly better than QLoRA **r32** (4.6158 versus 4.6376). The higher-rank condition therefore appears to trade a small Layer A regression for a stronger Layer B gain, leaving the overall result almost unchanged.

Taken together, these results suggest a simple capacity story. Increasing adapter rank from 16 to 32 is useful within the quantised regime under the shared `protocol_v3` settings, because it recovers quality at only a modest additional systems cost. Increasing rank further to 64 does not produce a commensurate overall gain. For the purposes of this dissertation, QLoRA **r32** is the more efficient higher-capacity quantised setting.

4.2.3 Learning-Curve Behaviour

Figure 4.2 adds the temporal view that Table 4.1 and Figure 4.1 cannot: it shows when the quality differences emerge, not just where they end. All four conditions realise most of their total gain early, and the best-checkpoint rank ordering is already visible well before the final evaluation step.

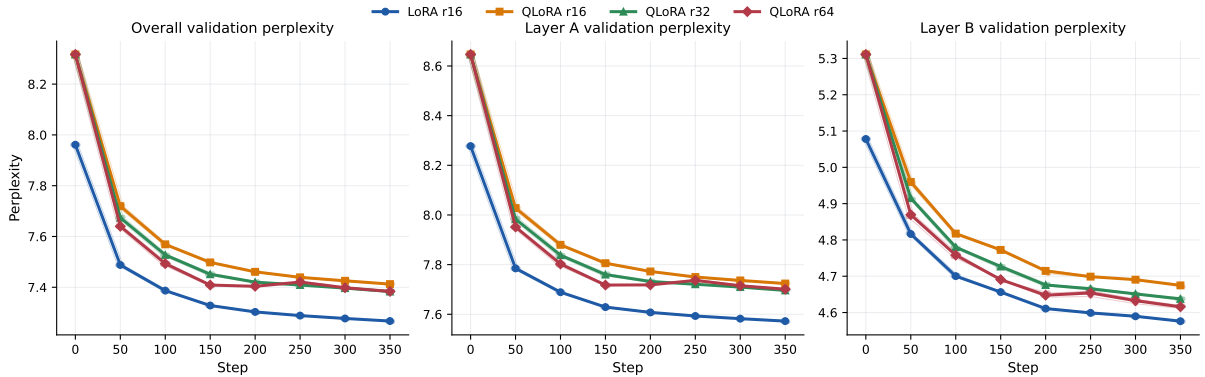


Figure 4.2: Validation perplexity curves across the shared seed set for the three reported views: overall, Layer A, and Layer B. Faint trajectories and points show individual seed runs, with a small horizontal jitter for visibility; bold lines show cross-seed means. The point at step 0 is the baseline-first held-out evaluation before adaptation.

The first 50 optimiser steps account for the largest quality improvement in every condition: overall perplexity falls to 7.4878 for LoRA **r16**, 7.7195 for QLoRA **r16**, 7.6740 for QLoRA **r32**, and 7.6401 for QLoRA **r64**. Later progress continues but slows, supporting the Chapter 2 claim that 350 steps capture the main observed intrinsic gains within this monitored window, without implying that longer schedules are exhausted.

After the initial baseline gap, LoRA **r16** remains the best overall condition at every evaluation point from step 50 onwards. The Phase 1 result is therefore not an artifact of one late checkpoint choice; the unquantised LoRA condition holds a persistent lead across the monitored trajectory. Within QLoRA, both **r32** and **r64** consistently outperform **r16** after the early stages, while **r32** and **r64** remain too close overall to separate confidently. The layer-specific views reinforce the

best-checkpoint result: **r64** appears to buy a specialised Layer B gain without a stronger overall outcome than **r32**.

4.3 Efficiency and Stability Trade-Offs

4.3.1 Memory and Runtime Trade-Offs

The systems-side results show a much clearer advantage for QLoRA than the quality results do. Using train peak allocated VRAM as the primary memory headline, **LoRA r16** averages 15.126 GB, whereas **QLoRA r16** averages 7.437 GB. Quantising the frozen base model therefore reduces peak training memory by 7.689 GB, or approximately 50.8%, in the core matched-rank comparison. This is the strongest practical benefit of QLoRA in the final run set, and it is fully consistent across seeds because the paired seed-aligned memory difference is -7.6886 GB in all three matched comparisons.

Figure 4.3 brings the central systems trade-off into view by plotting best overall perplexity against both train peak VRAM and mean step time. Read together with Table 4.1, it shows that the quantised conditions cluster in a distinct part of the trade-off space: much lower memory demand than **LoRA r16**, but substantially slower runtime and persistently worse primary-endpoint quality.

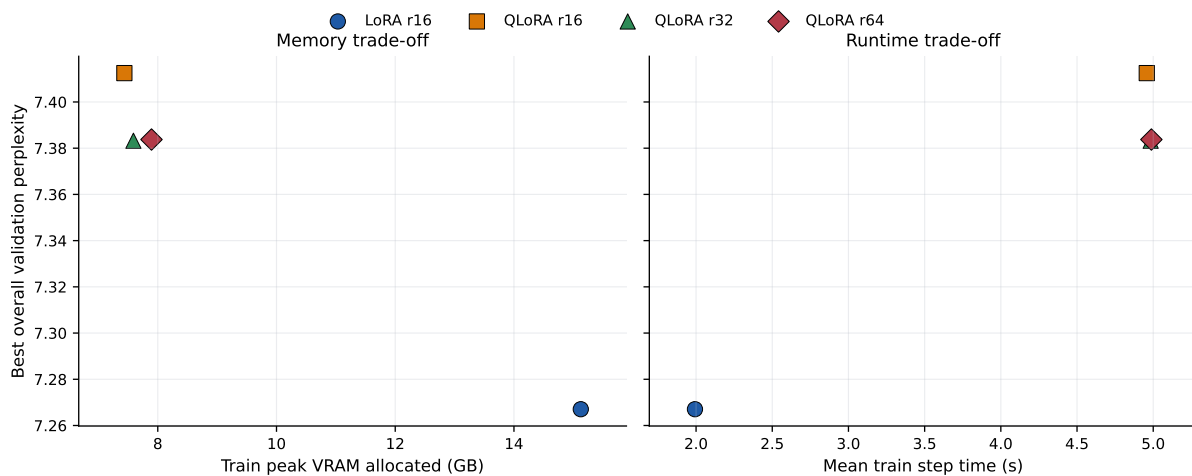


Figure 4.3: Trade-off views for the four completed conditions. Left: train peak VRAM versus best overall validation perplexity. Right: mean step time versus best overall validation perplexity.

That memory saving is accompanied by a substantial runtime penalty. Mean train step time rises from 1.993 s for **LoRA r16** to 4.958 s for **QLoRA r16**, and mean total runtime rises from 1088.7 s to 2688.9 s. Across seeds, **QLoRA r16** is slower by 2.95–2.98 s per measured training step. The core Phase 1 result is therefore not that QLoRA is simply “more efficient”, but that it exchanges a large memory saving for a large runtime cost while still failing to match LoRA’s best intrinsic quality.

The likely explanation is implementation-level. QLoRA stores the frozen base weights in 4-bit form, but training still runs through `bitsandbytes` quantised linear kernels, with

dequantisation or casting around matrix multiplications and packing overheads. The LoRA baseline instead uses unquantised `bfloat16` linear layers on the same L40S platform. Since the study did not include kernel-level profiling, this is a bounded systems interpretation rather than a universal claim about all QLoRA implementations[9, 20].

The QLoRA rank sweep shows that these systems costs remain comparatively stable once the model is already quantised. Moving from `r16` to `r32` raises mean train peak VRAM only from 7.437 GB to 7.590 GB and mean step time from 4.958 s to 4.983 s, while lowering mean best overall perplexity from 7.4125 to 7.3832. By contrast, `r64` leaves overall perplexity effectively tied with `r32` (7.3837 versus 7.3832), while mean train peak VRAM rises to 7.894 GB and mean step time to 4.988 s. The clearer systems conclusion is therefore narrow: QLoRA substantially reduces training memory, but within the quantised regime `r32` is the stronger quality–resource compromise than `r64` under the same 350-step comparison.

4.3.2 Stability Outcomes

On the formal stability definition adopted in Chapter 2, the final run set is uniformly stable. Across all twelve completed runs, no NaN loss, Inf loss, non-finite gradient norm events, OOM events, optimiser resets, or early terminations were recorded. Stability therefore does not differentiate the four reportable conditions. This conclusion is narrow: softer properties such as noisier loss curves remain outside the counted schema, and development-time OOMs on the consumer RTX 4070 belong to Chapter 3 rather than the final L40S evidence base.

4.4 Discussion and Answers to the Research Questions

4.4.1 Answer to the Research Questions

The discussion rests on the completed `protocol_v3` evidence base: twelve intrinsic runs, four reportable conditions, three shared seeds, one fixed document split, 350 optimiser steps, matched checkpoint selection, and the same runtime logging schema. Across those controls, the research questions converge on a trade-off result. For intrinsic quality, the matched-rank comparison favours LoRA `r16`: QLoRA `r16` improves substantially from its own quantised baseline, but does not catch the unquantised LoRA condition under the same corpus, optimiser exposure, and checkpoint rule. For systems cost, the answer reverses on memory but not on runtime: QLoRA gives the strongest memory saving, while also incurring a large step-time and wall-clock penalty. For stability, the final campaign records no objective failures, so stability does not differentiate the reportable conditions under the counted-event definition.

The rank-sweep question is best read inside that trade-off frame. Increasing QLoRA rank from 16 to 32 recovers most of the observed quantised rank-sweep quality gain while adding only about 0.153 GB of mean train peak VRAM and 0.025 s of mean step time. Increasing from 32 to 64 gives no meaningful additional overall gain while adding more memory. The dissertation therefore identifies QLoRA `r32` as the strongest quantised compromise in this run set, while still finding that it does not replace LoRA `r16` as the best-quality condition.

The overall answer to the dissertation aim is consequently scoped but clear. On this corpus, model family, L40S hardware platform, and 350-step adaptation schedule, 4-bit quantisation is not a free substitute for unquantised LoRA. It is a practical memory-saving intervention that changes the quality–runtime–memory balance of domain-adaptive PEFT. Different hardware, longer schedules, retuned hyperparameters, or downstream-task objectives could shift that balance, but they would be different studies from the matched intrinsic comparison reported here.

4.4.2 Relation to Prior Literature

The final results fit the literature positioned in Chapter 1 in three main ways. First, they support the continued relevance of domain-adaptive pretraining in a narrow domain setting: all four conditions improve substantially over their own baselines, which is consistent with the adaptation usefulness emphasised by Gururangan et al.[13] and Xie et al.[31]. Second, they refine the LoRA/QLoRA method story from Hu et al.[19] and Dettmers et al.[9]: the memory-saving side of QLoRA is strongly supported, but under the matched intrinsic comparison in this dissertation that saving does not imply parity with unquantised LoRA on held-out quality.

The results also remain compatible with the Chapter 1 claim that quantisation may change more than storage cost alone. The worse QLoRA baseline and the persistent post-adaptation quality gap are plausibly consistent with the transformer quantisation difficulties discussed by Dettmers et al.[8] and Bondarenko et al.[4]. In the legal domain, the chapter adds a narrower kind of evidence than work such as LEGAL-BERT, LexGLUE, or Pile of Law[5, 6, 14]: not downstream legal-task superiority, but a controlled account of how parameter-efficient intrinsic adaptation behaves inside a deliberately narrower practical legal subdomain when quality, memory, runtime, and stability are read together.

4.4.3 Limitations and Threats to Validity

The main limitations identified in Chapter 2 remain the right ones after seeing the final results. First, the chapter reports intrinsic held-out language-modelling behaviour only. This suits the dissertation aim, but it does not directly establish downstream legal-task performance, factual accuracy, citation quality, or practitioner usefulness.

Second, the evidence regime is bounded. All reportable results come from one fixed document-level train–validation split and three seeds per condition. That is enough to show consistent directional patterns through paired seed inspection, but it is not a high-powered inferential setting. Similarly, the 350-step schedule is justified by target-hardware pilot evidence and final curves, but it is not proven globally optimal for either method.

Third, the study uses one base-model family, one target hardware platform, one optimiser and schedule family, one attention-only adapter target pattern, and one 4-bit quantisation configuration. The resulting claims are strongest at the level actually studied: Meta-Llama-3-8B on the curated employment-law corpus under the `protocol_v3` recipe.

Two local interpretive constraints also remain. The QLoRA baseline uses a minor evaluation-path asymmetry because a purely quantised base model cannot be run through

exactly the same adapter-aware **Trainer** path before adapters are attached. In addition, packed-sequence evaluation is based on deterministic 1024-token chunks, so perplexity measures packed held-out intrinsic quality rather than whole-document coherence or discourse structure. These limitations set the boundary of the contribution. The evidence supports a controlled intrinsic trade-off characterisation under the fixed `protocol_v3` settings; it does not claim a universal ordering of LoRA and QLoRA across all legal tasks, hardware platforms, or training recipes.

4.5 Conclusions and Future Work

4.5.1 Chapter and Dissertation Conclusions

The dissertation set out to quantify how 4-bit quantisation changes parameter-efficient domain-adaptive continued pretraining when the data, optimisation opportunity, checkpoint rule, and reporting frame are fixed. The central contribution is a controlled characterisation of that trade-off surface. In this setting, quantisation makes the 8B adaptation path much lighter in memory, but not equal in intrinsic quality or faster in runtime.

The result is not a feasibility demonstration alone: it is a reproducible comparison with a fixed corpus, matched optimisation exposure, baseline-first evaluation, multi-seed aggregation, and systems outcomes reported alongside perplexity. The bounded conclusion is that QLoRA is viable and memory-efficient for this form of legal-domain adaptation, but not a cost-free replacement for unquantised LoRA under `protocol_v3`.

4.5.2 Future Work

The most direct extension is to widen the evidence regime while keeping the same comparative logic. More seeds, additional fixed splits, or cautious bootstrap intervals would strengthen descriptive confidence; a complementary study could then ask how much of the LoRA–QLoRA gap persists after modest method-specific retuning under a comparable resource cap.

A second extension is to connect the intrinsic results to downstream legal evaluation and broader systems settings: other legal tasks, base-model families, quantisation schemes, longer budgets, or adapter-target patterns may move the observed quality–memory–runtime boundary.

A targeted follow-up follows from the QLoRA rank sweep. Rank-stabilised LoRA (rsLoRA) uses a square-root-rank scaling variant intended to make larger ranks more useful[26]. Since this dissertation sees a useful `r16`-to-`r32` move but no clear `r64` benefit, rsLoRA would test whether that ceiling is task-specific or partly caused by standard scaling.

References

- [1] Acas. Code of practice on disciplinary and grievance procedures. Acas code of practice, 2015. Available at: <https://www.acas.org.uk/acas-code-of-practice-on-disciplinary-and-grievance-procedures/html>.
- [2] Acas. Advice. Acas guidance collection, 2026. Available at: <https://www.acas.org.uk/advice>.
- [3] Acas. Employment contracts and the law. Acas guidance, 2026. Available at: <https://www.acas.org.uk/employment-contracts-and-the-law>.
- [4] Y. Bondarenko, M. Nagel, and T. Blankevoort. Quantizable transformers: Removing outliers by helping attention heads do nothing. *arXiv preprint arXiv:2306.12929*, 2023.
- [5] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos. LEGAL-BERT: The muppets straight out of law school. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 2898–2904, Online, 2020. Association for Computational Linguistics.
- [6] I. Chalkidis, A. Jana, D. Hartung, M. Bommarito, I. Androutsopoulos, D. Katz, and N. Aletras. LexGLUE: A benchmark dataset for legal language understanding in english. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4310–4330, Dublin, Ireland, 2022. Association for Computational Linguistics.
- [7] Department for Business, Energy & Industrial Strategy. Employment status and employment rights. GOV.UK, 2022. Available at: <https://www.gov.uk/government/publications/employment-status-and-employment-rights>.
- [8] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. *arXiv preprint arXiv:2208.07339*, 2022.
- [9] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems 36*, 2023.
- [10] Government Digital Service. Employing people. GOV.UK guidance collection, 2026. Available at: <https://www.gov.uk/browse/employing-people>.
- [11] Government Digital Service. Working, jobs and pensions. GOV.UK guidance collection, 2026. Available at: <https://www.gov.uk/browse/working>.
- [12] A. Grattafiori et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [13] S. Gururangan, A. Marasovic, S. Swayamdipta, K. Lo, I. Beltagy, D. Downey, and N. A. Smith. Don’t stop pretraining: Adapt language models to domains and tasks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8342–8360, Online, 2020. Association for Computational Linguistics.

- [14] P. Henderson, M. S. Krass, L. Zheng, N. Guha, C. D. Manning, D. Jurafsky, and D. E. Ho. Pile of law: Learning responsible data filtering from the law and a 256gb open-source legal dataset. *arXiv preprint arXiv:2207.00220*, 2022.
- [15] HM Courts & Tribunals Service and Employment Tribunal. Employment tribunal decisions. GOV.UK, 2026. Available at: <https://www.gov.uk/employment-tribunal-decisions>.
- [16] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly. Parameter-efficient transfer learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2790–2799. PMLR, 2019.
- [17] House of Commons Library. Key employment rights. Research briefing, 2024. Available at: <https://commonslibrary.parliament.uk/research-briefings/cbp-7245/>.
- [18] House of Commons Library. All briefings. Research briefing collection, 2026. Available at: <https://commonslibrary.parliament.uk/research/all-research/>.
- [19] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- [20] Hugging Face. bitsandbytes. Hugging Face documentation, 2026. Available at: <https://huggingface.co/docs/bitsandbytes/en/index>.
- [21] Hugging Face. Causal language modeling. Hugging Face Transformers documentation, 2026. Available at: https://huggingface.co/docs/transformers/en/tasks/language_modeling.
- [22] Hugging Face. Lora. Hugging Face PEFT documentation, 2026. Available at: https://huggingface.co/docs/peft/main/en/conceptual_guides/lora.
- [23] Hugging Face. Quantization. Hugging Face PEFT documentation, 2026. Available at: https://huggingface.co/docs/peft/en/developer_guides/quantization.
- [24] Hugging Face. Trainer. Hugging Face Transformers documentation, 2026. Available at: https://huggingface.co/docs/transformers/en/main_classes/trainer.
- [25] Hugging Face. Training a causal language model from scratch. Hugging Face course documentation, 2026. Available at: <https://huggingface.co/docs/course/en/chapter7/6>.
- [26] D. Kalajdzievski. A rank stabilization scaling factor for fine-tuning with LoRA. *arXiv preprint arXiv:2312.03732*, 2023.
- [27] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- [28] A. Östling, H. Sargeant, H. Xie, L. Bull, A. Terenin, L. Jonsson, M. Magnusson, and F. Steffek. The cambridge law corpus: A dataset for legal AI research. In *Advances in Neural Information Processing Systems: Datasets and Benchmarks Track*, 2023.

- [29] PyTorch. Reproducibility. PyTorch documentation, 2026. Available at: <https://pytorch.org/docs/stable/notes/randomness.html>.
- [30] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, pages 5998–6008, 2017.
- [31] Y. Xie, K. Aggarwal, and A. Ahmad. Efficient continual pre-training for building domain specific large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10213–10227, 2024.

Appendix A

Self-appraisal

A.1 Critical self-evaluation

The project achieved its central technical aim: it produced a controlled, multi-seed intrinsic comparison of LoRA and QLoRA for domain-adaptive continued pretraining on a curated UK employment-law corpus. The strongest aspect of the work is that the final result is not based on a single run or on informal observation. The reportable `protocol_v3` campaign froze the corpus, split, sequence length, optimiser settings, runtime policy, step budget, evaluation cadence, checkpoint-selection rule, and training seeds before the final run matrix was completed. Each of the four reportable conditions was then run across the same three seeds and aggregated from structured run artefacts. This made the dissertation’s main conclusion more defensible: QLoRA was substantially more memory-efficient in the studied setting, but it was slower and did not match the best held-out intrinsic quality of unquantised LoRA.

A second strength was the development of the experimental infrastructure around the scientific question. The project began as a practical PEFT training pipeline, but the final implementation records considerably more than a final loss value. It emits resolved configurations, environment snapshots, dataset summaries, baseline metrics, evaluation curves, memory summaries, timing summaries, stability summaries, and comparison rows. This mattered because the project’s research question was multi-objective. Without structured reporting, memory, runtime, and stability would have been much easier to treat as secondary anecdotes rather than as evidence in their own right.

The corpus work was also a meaningful contribution. The final corpus was not a generic legal dataset copied into the project; it was constructed around a specific legal subdomain and split into tribunal judgments and guidance/doctrine layers. That made the study better aligned with the dissertation title than the earlier Eur-Lex direction. The reset from the initially larger tribunal scrape to a controlled token budget was a difficult but correct decision. It reduced the amount of data available, but it made the eventual comparison easier to interpret because the corpus and split were fixed before the reportable training runs.

The main weakness of the project is that its evaluation scope is narrower than the original ambition. Earlier versions of the plan included downstream classification-style evaluation, but the final dissertation focuses on intrinsic held-out language-modelling quality. I think this was the right trade-off by the end of the project, because the intrinsic comparison became substantially stronger once the protocol, logging, and multi-seed design were stabilised. However, it does mean that the dissertation cannot claim that the adapted models are better legal reasoning systems, better legal assistants, or more useful to practitioners. It can only claim what it measured: behaviour under the causal language-modelling objective on held-out text from the constructed corpus.

There are also limitations in the evidence regime. The final campaign uses one fixed document-level split and three training seeds per condition. This is a reasonable compromise for an undergraduate dissertation with constrained compute, and the seed-aligned results are directionally consistent, but it is not a high-powered statistical study. Similarly, the final 350-step budget was chosen from direct pilot evidence rather than from an exhaustive schedule search. This improves fairness within the final comparison, but it does not prove that either LoRA or QLoRA was globally optimised.

In retrospect, I would also have frozen the reporting schema earlier. Some of the most valuable engineering work happened after it became clear that terminal logs and ad hoc summaries were not enough for the kind of comparison the report needed to make. Designing the artefact contract earlier would have reduced rework and made the transition from exploratory experiments to reportable runs smoother. Even so, the final pipeline benefited from that lesson: the completed `protocol_v3` run set is reproducible from versioned configs and can be audited from saved structured outputs.

A.2 Personal reflection and lessons learned

The most important lesson I learned was the value of narrowing a research question until it becomes genuinely answerable. At the start of the project, it was tempting to treat domain adaptation, legal NLP, PEFT, quantisation, downstream transfer, and systems benchmarking as one combined problem. That scope was too broad. The project improved when I accepted that a smaller but more controlled question would produce a better dissertation than a larger question answered loosely.

The scope pivot from Eur-Lex to UK employment law was an early example of this. Eur-Lex was useful for validating the pipeline, but it was not well matched to the final motivation. Moving to UK employment-law text made the work more coherent and more original, but it also forced me to build a corpus pipeline and make sampling decisions carefully. Discarding the first large tribunal scrape was frustrating, because it represented real work, but it was methodologically necessary once the study became a controlled LoRA–QLoRA comparison. That experience taught me that sunk effort is not a good reason to keep a design choice that no longer supports the research question.

I also learned that experimental governance is not just administrative overhead. The progression from earlier protocols to `protocol_v3` helped distinguish exploration from final evidence. This was particularly important after the March target-hardware pilots showed that both LoRA and QLoRA reached their best observed validation perplexity around step 350 under the earlier longer schedule. Freezing a shorter final protocol and reallocating compute to three seeds per condition was a better scientific choice than continuing to tune after seeing partial results.

The cluster work was another major learning experience. Several failures were not scientific failures: a stale model identifier caused an early loading failure, the QLoRA baseline needed a manual evaluation path, one quantised baseline attempt exposed an attention/kernel issue, and later jobs appeared to stall because of shared Hugging Face cache lock contention. It would have been easy to treat these as uninteresting implementation details, but they affected whether the

experiment could be completed at all. The lesson was to separate recipe failures from infrastructure failures, document both, and avoid changing scientific constants merely because the launch path needed repair.

I became more disciplined about evidence as the project progressed. Earlier in the project, I was more inclined to inspect a run manually and then write down what seemed important. By the final campaign, the pipeline produced comparison rows, condition summaries, and plot-ready evaluation curves automatically. This changed the character of the work: the results chapter could be written from a stable evidence layer rather than from memory or selective inspection.

If I were starting again, I would make three process changes. First, I would define the final evaluation semantics much earlier, especially the distinction between intrinsic validation perplexity and downstream legal utility. Second, I would design the run artefact schema before running any expensive experiments. Third, I would reserve more time at the end specifically for report compression. The final report has a strong technical core, but writing it into a concise 30-page body is itself a substantial task and should have been planned as a major work package rather than as a final formatting pass.

A.3 Legal, social, ethical and professional issues

A.3.1 Legal issues

The project used external legal text, external software, and an externally released base model, so legal and attribution issues were relevant even though the project did not deploy a public system. The employment tribunal judgments and guidance material used for the corpus are public sources, but public availability does not remove the responsibility to describe their origin accurately and avoid implying ownership of the source material. The report therefore treats corpus construction, filtering, preprocessing, splitting, and analysis as the project contribution, while the underlying public legal documents remain external material.

The model and software stack also required care. The base model, **Meta-Llama-3-8B**, is an externally released model rather than work created for this dissertation. The implementation also depends on external libraries such as PyTorch, Transformers, PEFT, bitsandbytes, and Datasets. These dependencies must be acknowledged in Appendix B and in the references where appropriate. The professional standard here is not merely to cite papers, but to make clear which parts of the deliverable were developed in the project and which were built on existing models, libraries, and public data.

A.3.2 Social issues

The main social risk is overinterpretation. Legal-domain language models can be misunderstood as legal advice tools, especially when the domain is familiar and socially consequential. This dissertation does not create or evaluate a system for advising workers, employers, lawyers, or tribunal users. It evaluates intrinsic language-modelling behaviour under a controlled adaptation protocol. For that reason, the report deliberately avoids claiming practitioner utility from lower

perplexity alone.

The employment-law setting also deserves care because the source texts relate to workplace disputes, discrimination, dismissal, wages, disability, reasonable adjustments, and other issues that can have serious consequences for real people. Even when such judgments are public, they are not socially neutral text. A model trained on this domain could reproduce patterns in legal and institutional language without understanding the human context behind them. This reinforces the importance of presenting the project as a methodological study of adaptation trade-offs rather than as a ready route to automated legal decision-making.

A.3.3 Ethical issues

The project did not involve human participants, user studies, private interviews, or collection of new personal data, so consent protocols were not required. Ethical issues were nevertheless present because the corpus includes public legal decisions that may describe sensitive workplace events. The project reduced this risk by using the documents for aggregate intrinsic evaluation rather than by publishing generated case summaries, exposing individual examples unnecessarily, or building an interactive legal advice system.

There is also an ethical obligation to report limitations clearly. A lower held-out perplexity score means that a model assigns higher average probability to tokens in the validation corpus. It does not mean that the model is fair, safe, legally reliable, or suitable for use by vulnerable users. The report’s limitations sections and this self-appraisal therefore avoid translating an intrinsic metric into a claim about legal competence.

A.3.4 Professional issues

The main professional responsibility in this project was to make the work auditable and not overclaim the findings. This meant keeping versioned protocols, preserving development logs, freezing the final experimental constants, recording failed attempts where they mattered, and separating pilot evidence from final reportable evidence. It also meant resisting the temptation to retune the recipe after partial results were visible.

Professional conduct also required a clear account of reproducibility. The final runs are tied to versioned configuration files, structured artefacts, and environment snapshots. This does not make the project perfectly reproducible on all hardware, because the final campaign depends on a particular GPU platform and software stack, but it does make the reported results recoverable and auditable from the project repository.

Finally, the project raised a professional communication issue. Because legal AI is a sensitive area, the report needed to be precise about what was and was not demonstrated. The strongest defensible contribution is a bounded empirical trade-off study: under the fixed `protocol_v3` settings, QLoRA reduced memory, increased runtime, and remained worse than LoRA on held-out intrinsic quality. Stating that result carefully is part of the technical contribution, because imprecise or inflated claims would make the work less useful to future readers.

Appendix B

External Material

The project repository submitted as a deliverable is available at:

<https://github.com/gsleeds/llama-peft-project>. The table below records the main external materials used in the project and distinguishes them from the work developed for the dissertation.

The main public corpus source pages were GOV.UK employment tribunal decisions (<https://www.gov.uk/employment-tribunal-decisions>), ACAS advice (<https://www.acas.org.uk/advice>), GOV.UK employing-people guidance (<https://www.gov.uk/browse/employing-people>), GOV.UK working guidance (<https://www.gov.uk/browse/working>), and House of Commons Library briefings (<https://commonslibrary.parliament.uk/>).

External material	Origin	Role in the project	Student-developed work
Base model	meta-llama/ Meta-Llama-3-8B, released by Meta and accessed through Hugging Face	Starting pretrained causal language model for the LoRA and QLoRA adaptation experiments	Selection, configuration, PEFT adaptation setup, evaluation protocol, and interpretation of results
Employment tribunal decisions	GOV.UK employment tribunal decisions	Source material for Layer A of the UK employment-law corpus	Scraping/downloading workflow, PDF-to-text conversion, cleaning, filtering, reflow, token accounting, document selection, split creation, and corpus freeze
ACAS employment guidance and codes	ACAS advice and code pages, including material under https://www.acas.org.uk/advice	Source material for Layer B guidance and practical employment-law text	Manual source selection, extraction, cleaning, source tagging, integration with Layer A, and corpus statistics
GOV.UK employment and working guidance	GOV.UK pages under https://www.gov.uk/browse/employing-people and https://www.gov.uk/browse/working	Additional Layer B public guidance on employment rights and workplace topics	Selection, preprocessing, cleaning, and integration into the fixed corpus
House of Commons Library briefings	House of Commons Library research briefings from https://commonslibrary.parliament.uk/	Additional Layer B doctrinal and policy-context material	Selection, extraction, cleaning, and integration into the fixed corpus

Table B.1: External model and corpus materials used in the dissertation.

External material	Origin	Role in the project	Student-developed work
Python and machine-learning libraries	PyTorch, Transformers, PEFT, bitsandbytes, Datasets, Accelerate, Hugging Face Hub, tokenizers, safetensors, NumPy, pandas, PyYAML, requests, BeautifulSoup, pdfplumber, tqdm, and related dependencies listed in <code>requirements.txt</code> and <code>environment.yaml</code>	Provided standard APIs for model loading, tokenisation, LoRA/QLoRA configuration, 4-bit quantisation, training, data handling, scraping, PDF extraction, and reporting	Project-specific preprocessing scripts, training pipeline, callbacks, manual QLoRA baseline evaluation path, memory/timing/stability instrumentation, aggregation scripts, and report artefact generation
CUDA, Slurm, and cluster environment	NVIDIA CUDA/PyTorch GPU stack and the university cluster job environment	Execution environment for local calibration, target-hardware smoke tests, and final L40S run campaign	Cluster launchers, preflight checks, per-job cache handling, protocol-specific Slurm job script, and failure-handling workflow
Technical documentation	Official PyTorch, Hugging Face Transformers, PEFT, bitsandbytes, and Hugging Face course documentation cited in the bibliography	Informed use of standard APIs, causal language-modelling data preparation, Trainer configuration, quantisation settings, LoRA/QLoRA setup, and reproducibility controls	The dissertation code was written for this project; documentation was used as API and method reference material rather than as copied project code
Academic literature	Papers and resources cited in the bibliography, including work on transformers, DAPT, LEGAL-BERT, LexGLUE, LoRA, QLoRA, quantisation, and legal corpora	Provided background, method definitions, motivation, and comparison context	Literature synthesis, research gap framing, methodology design, and interpretation of the dissertation’s empirical results

Table B.2: External software, documentation, and literature used in the dissertation.

No external project-specific source code was copied into the submitted implementation. The software deliverable was written for this dissertation using the external libraries and documentation listed above.